

Zella Luxe

Technical Documentation

Luxury Women's E-Commerce Platform for Algeria

<https://zella-luxe.vercel.app>

French + Arabic | Next.js 16 | PostgreSQL | Vercel

Live URL	https://zella-luxe.vercel.app
Version	0.1.0
Document date	July 2026
Languages	French (LTR) + Arabic (RTL)

Table of Contents

- 1. Project Overview
- 2. Technology Stack
- 3. Architecture Overview
- 4. Project Structure
- 5. Database Schema
- 6. Frontend Features
- 7. Admin Panel
- 8. API Routes Reference
- 9. Key Code Patterns
- 10. Internationalization (i18n)
- 11. Design System
- 12. Deployment & Operations
- 13. Data Flow Diagrams
- 14. Environment Variables
- 15. Appendix: Key File Index

1. Project Overview

1.1 What is Zella Luxe?

Zella Luxe is a full-stack e-commerce website for a luxury women's fashion boutique operating in Algeria. The store specializes in:

- Shoes — heels (talons), escarpins, sandals, mules, ballerinas, mocassins, boots (bottes), sneakers, and more
- Bags — handbags (sacs) and clutches (pochettes)
- Luggage — premium suitcases (valises)

The brand positions itself with an elegant, gold-and-black aesthetic targeting Algerian customers with nationwide delivery across all 58 wilayas (provinces).

1.2 Business Model

Aspect	Implementation
Payment	Cash on delivery (paiement à la livraison)
Order confirmation	Database record + pre-formatted Instagram message for the shop owner
Customer reach	Instagram, Facebook, TikTok, WhatsApp integration
Physical stores	Annaba and Guelma (contact page with maps & phone numbers)

1.3 Live Deployment

- Production URL: <https://zella-luxe.vercel.app>
- Hosting: Vercel (serverless Next.js)
- Database: Neon PostgreSQL (managed, serverless)
- Image uploads (reviews): Cloudinary CDN

1.4 Supported Languages

Language	Code	Direction	Font
French	fr	LTR (left-to-right)	Jost (body), Playfair Display (headings)
Arabic	ar	RTL (right-to-left)	Noto Sans Arabic

Users switch language via a toggle in the header. Preference is persisted in localStorage under key zella-luxe-lang.

2. Technology Stack

2.1 Core Framework

Technology	Version	Role
Next.js	16.2.10	App Router, SSR/SSG, API routes
React	19.2.4	UI components
TypeScript	5.x	Type safety across the codebase
Tailwind CSS	4.x	Utility-first styling via @import "tailwindcss"

2.2 Backend & Data

Technology	Version	Role
Prisma ORM	6.19.3	Database access, schema management, migrations
PostgreSQL	—	Primary data store (Neon in production)
Zod	4.4.3	Runtime validation (where used)
bcryptjs	3.0.3	Optional bcrypt password comparison for admin

2.3 Infrastructure & Services

Service	Purpose
Vercel	Hosting, CI/CD on git push, serverless functions
Neon	Managed PostgreSQL (DATABASE_URL)
Cloudinary	Review photo uploads in production
Docker Compose	Optional local PostgreSQL for development

2.4 Client-Side Persistence

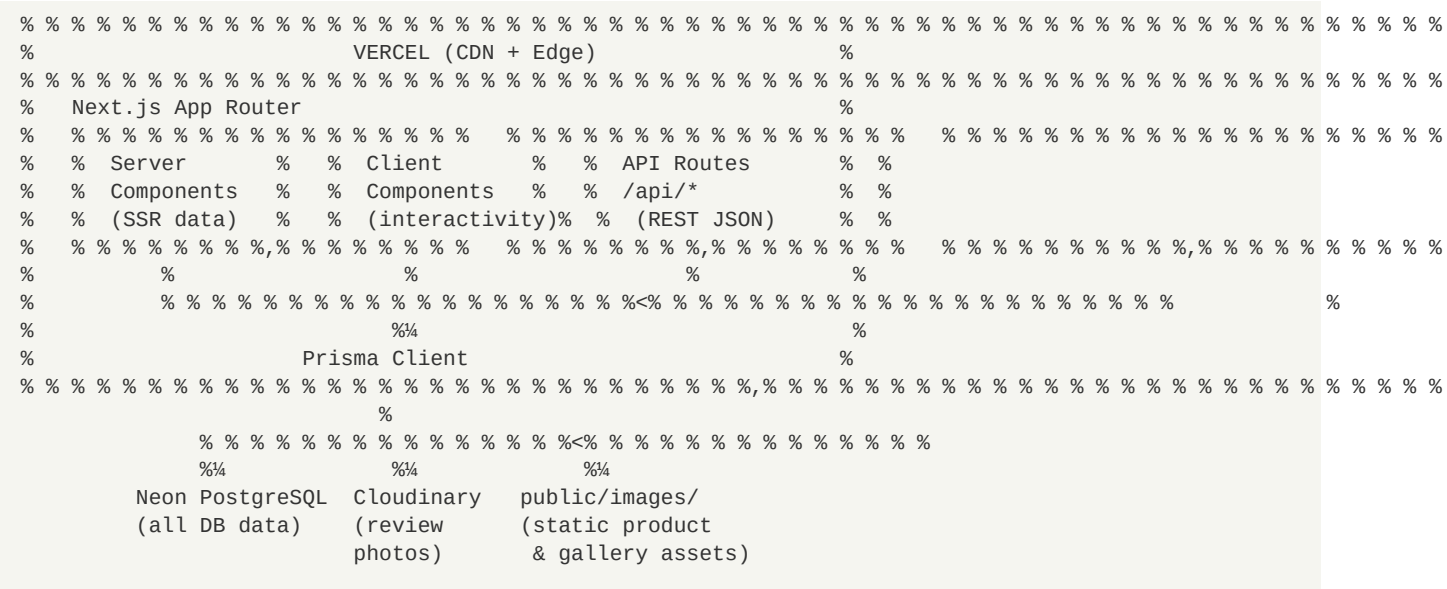
Data	Storage	Key
Shopping cart	localStorage	zella-luxe-cart
Wishlist / Favoris	localStorage	zella-luxe-wishlist
Language preference	localStorage	zella-luxe-lang
Admin session	HTTP-only cookie	zella_admin_session

2.5 UI Libraries

- lucide-react — Icon set (navigation, admin, product actions)
- next/font/google — Playfair Display, Jost, Noto Sans Arabic, Great Vibes

3. Architecture Overview

Zella Luxe follows a monolithic Next.js full-stack architecture:



Request flow (typical page):

1. User visits a route (e.g. /catalogue)
2. Server Component fetches data from Prisma
3. Data passed to Client Component for interactivity
4. Client mutations call /api/* routes
5. API routes validate, write to DB, return JSON

4. Project Structure

4.1 Root Directory

```
zella-luxe/  
% % % prisma/  
%   % % % schema.prisma      # Database models  
%   % % % seed.ts           # Initial products, gallery, settings  
%   % % % shipping-seed.ts   # 58 wilayas + delivery offices  
% % % public/  
%   % % % images/           # Static product & design images  
% % % scripts/              # Utility scripts (wilaya checks)  
% % % src/  
%   % % % app/              # Next.js App Router (pages + API)  
%   % % % components/       # Reusable UI components  
%   % % % context/          # React Context providers  
%   % % % lib/              # Utilities, auth, shipping, i18n  
% % % .env.example          # Environment variable template  
% % % DEPLOY-VERCEL.md       # Deployment guide (French)  
% % % next.config.ts         # Next.js configuration  
% % % package.json          # Dependencies & scripts  
% % % vercel.json            # Vercel service routing  
% % % docker-compose.yml     # Local PostgreSQL
```

4.2 src/app/ — Pages & Routes

Path	File	Description
/	page.tsx	Homepage (SSR: featured, promos, gallery, reviews)
/catalogue	catalogue/page.tsx	Full product catalogue with filters
/produit/[slug]	produit/[slug]/page.tsx	Product detail page
/promotions	promotions/page.tsx	Products on promotion
/panier	panier/page.tsx	Cart + checkout
/favoris	favoris/page.tsx	Wishlist page
/avis	avis/page.tsx	Customer reviews (submit + list)
/galerie	galerie/page.tsx	Photo gallery / lookbook
/contact	contact/page.tsx	Contact form + store locations
/admin/*	admin/**	Admin panel (protected)
/api/*	api/**/route.ts	REST API endpoints

4.3 src/components/ — UI Components

Component	File	Purpose
Header	Header.tsx	Top navigation, language toggle, cart badge
Footer	Footer.tsx	Social links, copyright
Hero	Hero.tsx	Homepage hero banner
TrustBar	TrustBar.tsx	Trust badges (delivery, payment, quality)
HomeClient	HomeClient.tsx	Homepage sections (categories, featured, lookbook)
ProductCard	ProductCard.tsx	Product grid card (mobile 2-up layout)
QuickView	QuickView.tsx	Modal quick-view for products
CheckoutForm	CheckoutForm.tsx	Checkout form with wilaya/shipping
OrderSummary	OrderSummary.tsx	Cart totals sidebar
ReviewsSection	ReviewsSection.tsx	Review display on homepage
SplashScreen	SplashScreen.tsx	Animated welcome splash (skipped on admin)
MobileBottomNav	MobileBottomNav.tsx	Fixed bottom nav on mobile
LanguageToggle	LanguageToggle.tsx	FR/AR switcher
SocialLinks	SocialLinks.tsx	Instagram, Facebook, TikTok, WhatsApp icons
Providers	Providers.tsx	Wraps all context providers + layout shell
admin/*	admin/	Admin forms, nav, image/color/size fields

4.4 src/context/ — State Management

Context	File	Storage
CartContext	CartContext.tsx	localStorage + API sync for prices
WishlistContext	WishlistContext.tsx	localStorage (product IDs)
LanguageContext	LanguageContext.tsx	localStorage + document.dir

4.5 src/lib/ — Shared Utilities

Module	File	Purpose
prisma	prisma.ts	Singleton Prisma client
auth	auth.ts	Admin session cookie management
i18n	i18n.ts	French/Arabic translation strings
categories	categories.ts	Product category enums & labels
shipping	shipping.ts	Client-side shipping cost helpers
shipping-server	shipping-server.ts	Server-side wilaya/office data loader
instagram	instagram.ts	Order message builder, handle validation
social	social.ts	Social link resolution with defaults
settings	settings.ts	Load SiteSettings from DB
cloudinary	cloudinary.ts	Cloudinary signed upload for reviews
uploads	uploads.ts	Local vs Cloudinary upload strategy
images	images.ts	Design-only image exclusions
utils	utils.ts	Price formatting, JSON parsing, slugs
algeria	algeria.ts	Algeria-specific helpers

5. Database Schema

The database is defined in `prisma/schema.prisma` and uses PostgreSQL via Prisma ORM.

5.1 Entity Relationship Diagram

```
Product % % % % % < Review
%
% (optional productId)

Wilaya % % % % % < DeliveryOffice

SiteSettings (singleton, id=1)
ContactMessage (standalone)
Order (standalone, items as JSON)
GalleryImage (standalone)
```

Note: There is no Admin database model. Admin authentication uses the `ADMIN_PASSWORD` environment variable and an HTTP-only session cookie.

5.2 Model: Product

The core catalogue entity. Bilingual fields support French and Arabic.

Field	Type	Description
id	String (cuid)	Primary key
slug	String (unique)	URL-friendly identifier (/produit/[slug])
nameFr / nameAr	String	Product name in both languages
descriptionFr / descriptionAr	String	Full descriptions
price	Float	Regular price in Algerian Dinars (DA)
promoPrice	Float?	Discounted price when on promotion
onPromo	Boolean	Whether product is currently promoted
category	String	One of 14 categories (see §5.8)
images	String (JSON)	Array of image URLs, e.g. ["/images/foo.jpg"]
colors	String (JSON)	Array of {nameFr, nameAr, hex?, image?} objects
sizes	String (JSON)	Array of size strings, e.g. ["36","37","38"]
stock	Int	Available quantity (decremented on order)
featured	Boolean	Show on homepage featured section
active	Boolean	Visible in catalogue (soft hide when false)
createdAt / updatedAt	DateTime	Timestamps

Relationships: One Product has many Reviews.

5.3 Model: Order

Stores customer orders placed through checkout.

Field	Type	Description
id	String (cuid)	Primary key
orderNumber	String (unique)	Human-readable order ID
customerName	String	Full name (firstName + lastName)
firstName / lastName	String	Split name fields
phone	String	Algerian mobile number
email	String?	Optional email
address	String	Formatted delivery address string
wilayaCode / wilayaName	String	Selected province
deliveryMethod	String	HOME or OFFICE
deliveryAddress	String?	Full address (home delivery only)
deliveryOfficeId / deliveryOfficeName	String?	Pickup office (office delivery only)
instagramHandle	String?	Customer's Instagram handle
items	String (JSON)	Cart items snapshot
subtotal	Float	Products total
shippingCost	Float	Delivery fee
total	Float	Grand total
status	String	Default PENDING (admin can update)
notes	String?	Customer notes

5.4 Model: Wilaya

Algeria's 58 provinces with shipping rates.

Field	Type	Description
id	String (cuid)	Primary key
code	String (unique)	Wilaya code (e.g. "23" for Annaba)
nameFr / nameAr	String	Province name
homeShippingPrice	Float	Home delivery cost (default 600 DA)
officeShippingPrice	Float	Office pickup cost (default 400 DA)
active	Boolean	Available for checkout

Relationships: One Wilaya has many DeliveryOffices.

5.5 Model: DeliveryOffice

Pickup locations for office delivery method.

Field	Type	Description
id	String (cuid)	Primary key
wilayald	String	Foreign key to Wilaya
nameFr / nameAr	String	Office name
active	Boolean	Available for selection

5.6 Model: Review

Customer-submitted product reviews with moderation workflow.

Field	Type	Description
id	String (cuid)	Primary key
customerName	String	Reviewer name
instagramHandle	String?	Optional Instagram
phone	String?	Optional phone
productId	String?	Optional linked product
message	String	Review text
rating	Int	1–5 stars
photos	String (JSON)	Array of image URLs (Cloudinary)
status	String	PENDING !' APPROVED or REJECTED

5.7 Model: GalleryImage

Lookbook / gallery photos managed by admin.

Field	Type	Description
id	String (cuid)	Primary key
url	String	Image URL
captionFr / captionAr	String?	Optional captions
sortOrder	Int	Display order
active	Boolean	Visible on site

5.8 Model: ContactMessage

Messages from the contact form.

Field	Type	Description
id	String (cuid)	Primary key
name	String	Sender name
instagramHandle	String?	Optional Instagram
phone	String?	Optional phone
message	String	Message body
read	Boolean	Admin read status

5.9 Model: SiteSettings

Singleton configuration row (id = 1).

Field	Type	Default	Description
instagramUrl	String	""	Instagram profile URL
facebookUrl	String	""	Facebook page URL
tiktokUrl	String	""	TikTok profile URL
whatsapp	String	""	WhatsApp phone number
phone	String	""	Contact phone
email	String	contact@zellaluxe.dz	Contact email
addressFr / addressAr	String	Algérie / bVDbÆ2bv&c	Business address

Social links fall back to hardcoded defaults in `src/lib/social.ts` when DB values are empty.

5.10 Product Categories

Defined in `src/lib/categories.ts`:

Category	French Label
TALONS	Talons
ESCARPINS	Escarpins
SANDALES	Sandales
MULES	Mules
BALLERINES	Ballerines
MOCASSINS	Mocassins
SABOTS	Sabots
DERBIES	Derbies
BASKETS	Baskets
BOTTES	Bottes
CHAUSSURES	Chaussures
SACS	Sacs
POCHETTES	Pochettes
VALISES	Valises

Each category has a unique gold accent color used in admin pills and homepage category cards.

6. Frontend Features

6.1 Homepage (/)

Server: `src/app/page.tsx` — fetches featured products, promotions, gallery images, and approved reviews from Prisma.

Client: `src/components/HomeClient.tsx` renders:

Section	Description
Hero	Full-width banner with brand tagline and CTAs
TrustBar	Four trust badges (58 wilayas, COD, exchange, quality)
WelcomeSection	Boutique welcome message
Promotions	Up to 4 products with <code>onPromo: true</code>
Categories	Grid of all 14 product categories with accent colors
Featured Products	Up to 4 products with <code>featured: true</code>
Reviews	Latest 6 approved customer reviews
Lookbook / Gallery	Up to 8 active gallery images

Supporting files: `Hero.tsx`, `TrustBar.tsx`, `WelcomeSection.tsx`, `ReviewsSection.tsx`, `ProductCard.tsx`

6.2 Catalogue (/catalogue)

Files: `catalogue/page.tsx`, `catalogue/CatalogueClient.tsx`

Feature	Implementation
Search	Client-side filter by product name (FR/AR)
Category filter	Filter chips for all 14 categories
Sort	Newest, price ascending/descending, promotions
Pagination	"Load more" button (incremental reveal)
Product cards	<code>ProductCard</code> with ratings, promo badges, wishlist

6.3 Product Detail (/produit/[slug])

Files: `produit/[slug]/page.tsx`, `produit/[slug]/ProductDetailClient.tsx`

Feature	Implementation
Image gallery	Multiple images with color-based switching
Color swatches	ColorSwatches.tsx — select color variant
Size selection	Parsed from sizes JSON array
Stock indicator	Shows in-stock / out-of-stock
Add to cart	Via CartContext.addItem()
Wishlist toggle	Via WishlistContext.toggle()
Star ratings	Aggregated from approved reviews
Related products	Same category suggestions
Promo pricing	Shows original + discounted price with % badge

6.4 Cart & Checkout (/panier)

Files: panier/page.tsx, panier/PanierClient.tsx, CheckoutForm.tsx, OrderSummary.tsx

Cart features:

- View items with image, name, size, color, quantity
- Adjust quantity or remove items
- Subtotal calculated from current promo prices
- Persists in localStorage (zella-luxe-cart)

Checkout flow:

1. Customer fills: first name, last name, phone, optional Instagram
2. Selects wilaya (58 provinces loaded from API)
3. Chooses delivery method: Home or Office pickup
4. Home: enters full address; Office: selects delivery bureau
5. Optional notes field
6. Server validates phone (Algerian format), recalculates totals server-side
7. Order saved to database, stock decremented
8. Success screen with pre-formatted Instagram message to copy/send

Shipping cost calculation: getShippingCost() in src/lib/shipping.ts uses wilaya's homeShippingPrice or officeShippingPrice.

6.5 Wishlist / Favoris (/favoris)

Files: favoris/page.tsx

- Product IDs stored in localStorage (zella-luxe-wishlist)
- Page fetches product details via GET /api/products/by-ids?ids=...
- Toggle heart icon on any ProductCard

- Empty state with link to catalogue

6.6 Quick View Modal

File: src/components/QuickView.tsx

- Opened from eye icon on ProductCard
- Shows product image, price, size/color selectors
- Add to cart without leaving current page
- Link to full product detail page

6.7 Promotions (/promotions)

Files: promotions/page.tsx, promotions/PromotionsClient.tsx

- Lists all products where onPromo: true and active: true
- Shows discount percentage badge
- Same card layout as catalogue

6.8 Reviews / Avis (/avis)

Files: avis/page.tsx, avis/AvisClient.tsx

Public display:

- Lists approved reviews (status: "APPROVED")
- Shows star rating, customer name, message, photos

Submission form:

- Name, optional Instagram, optional phone
- Optional product selection
- 1–5 star rating
- Message text
- Photo upload (up to 5 MB, images only)
- Photos uploaded to Cloudinary (production) or local disk (dev)
- Review saved with status: "PENDING" — requires admin approval

6.9 Contact (/contact)

File: src/app/contact/page.tsx

Contact form:

- Name, optional Instagram, phone, message
- Submits to POST /api/contact
- Success confirmation state

Physical store locations (hardcoded):

City	Phone	Map
Annaba (c–Fbv(b'•	0775 31 02 35	Google Maps link
Guelma (d&'dFEb'•	0562 07 83 21	Google Maps link

6.10 Gallery (/galerie)

Files: galerie/page.tsx, galerie/GalerieClient.tsx

- Masonry-style image grid
- Images from GalleryImage model (active only)
- Bilingual captions
- Design-only background images excluded via DESIGN_ONLY_IMAGES

6.11 Splash Screen

File: src/components/SplashScreen.tsx

- Animated welcome overlay on first visit
- Brand logo, tagline, progress bar (~3.5 seconds)
- Skipped on /admin/* routes
- Uses CSS animations defined in globals.css

6.12 Mobile Navigation

File: src/components/MobileBottomNav.tsx

Fixed bottom navigation bar (visible on screens < lg):

Icon	Route	Label
Home	/	Accueil
Grid	/catalogue	Catalogue
Tag	/promotions	Promotions
Bag	/panier	Panier (with count badge)
Message	/contact	Contact

Hidden on admin routes. Respects safe-area-inset-bottom for notched phones.

6.13 Responsive Product Grid

File: src/components/ProductCard.tsx

- Mobile: 2-column grid layout with compact card design
- Desktop: Larger cards with hover effects, quick-view button
- Promo badges, "New" badge for recent products

- Star ratings and review count
- Color swatch preview dots

6.14 Social Links

Files: SocialLinks.tsx, src/lib/social.ts, src/lib/settings.ts

Platform	Source
Instagram	SiteSettings.instagramUrl !' fallback default
Facebook	SiteSettings.facebookUrl !' fallback default
TikTok	SiteSettings.tiktokUrl !' fallback default
WhatsApp	SiteSettings.whatsapp !' wa.me link

Loaded server-side in root layout and passed to Providers !' Header + Footer.

7. Admin Panel

7.1 Access & Authentication

Item	Detail
Login URL	/admin/login
Password source	ADMIN_PASSWORD environment variable
Session	HTTP-only cookie zella_admin_session (7-day expiry)
Dev fallback	zella2026 when ADMIN_PASSWORD unset in development
Logout	POST /api/admin/logout clears cookie

Auth flow:

1. Admin submits password on login page
2. POST /api/admin/login verifies against ADMIN_PASSWORD (plain text or bcrypt hash)
3. On success, createAdminSession() sets secure cookie
4. All admin pages and protected API routes check isAdminAuthenticated()
5. Unauthenticated access to /admin redirects to /admin/login

Files: src/lib/auth.ts, src/app/admin/login/page.tsx, src/app/api/admin/login/route.ts

7.2 Admin Dashboard (/admin)

File: src/app/admin/page.tsx

Dashboard cards with live counts:

Card	Route	Metric
Produits	/admin/produits	Total products
En promotion	/admin/promotions	Products on promo
Commandes en attente	/admin/commandes	Orders with status PENDING
Avis en attente	/admin/avis	Reviews with status PENDING
Messages non lus	/admin/messages	Unread contact messages
Images galerie	/admin/galerie	Gallery image count
Tarifs livraison	/admin/livraison	Manage wilaya shipping rates
Bureaux	/admin/bureaux	Manage delivery offices

7.3 Product Management (/admin/produits)

Files: admin/produits/page.tsx, AdminProductsClient.tsx, AdminProductForm.tsx

Feature	Detail
List all products	With category filter pills in nav
Create product	POST /api/products
Edit product	PUT /api/products/[id]
Delete product	DELETE /api/products/[id]
Fields	Names (FR/AR), descriptions, price, promo, category, images, colors, sizes, stock, featured, active
Category filter	14 categories including BOTTES, SACS, VALISES, etc.

Sub-components:

- AdminImageFields.tsx — Image URL management
- AdminColorFields.tsx — Color name + hex + optional image
- AdminSizeFields.tsx — Size array editor

7.4 Promotions (/admin/promotions)

File: admin/promotions/AdminPromotionsClient.tsx

- Lists products with onPromo: true
- Toggle promotion on/off
- Set promoPrice

7.5 Orders (/admin/commandes)

File: admin/commandes/AdminOrdersClient.tsx

- Lists all orders (newest first)
- View customer details, items, totals
- Update order status via PATCH /api/orders/[id]

7.6 Review Moderation (/admin/avis)

File: admin/avis/AdminReviewsClient.tsx

- Lists all reviews (including pending)
- Approve: PATCH /api/reviews/[id] with status: "APPROVED"
- Reject: status: "REJECTED"
- Delete review

7.7 Gallery Management (/admin/galerie)

File: admin/galerie/AdminGalleryClient.tsx

- Add/edit/delete gallery images
- Set captions (FR/AR), sort order, active status
- Blocks design-only background images

7.8 Shipping Rates (/admin/livraison)

File: admin/livraison/AdminShippingClient.tsx

- Manage all 58 wilayas
- Edit homeShippingPrice and officeShippingPrice per wilaya
- Toggle wilaya active/inactive
- API: GET/PATCH /api/admin/wilayas, PATCH /api/admin/wilayas/[id]

7.9 Delivery Offices (/admin/bureaux)

File: admin/bureaux/AdminOfficesClient.tsx

- CRUD delivery offices per wilaya
- API: GET/POST /api/admin/delivery-offices, PUT/DELETE /api/admin/delivery-offices/[id]

7.10 Contact Messages (/admin/messages)

File: admin/messages/page.tsx

- Lists contact form submissions
- Mark messages as read

7.11 Admin Layout & Navigation

Files: admin/layout.tsx, components/admin/AdminNav.tsx

- Light-themed admin shell (contrasts with dark storefront)
- Horizontal nav with all admin sections

- Category pills when on products page
- "Site" link back to storefront
- LogoutButton component

8. API Routes Reference

All API routes are under `src/app/api/` and return JSON.

8.1 Public Routes

Method	Route	Description
GET	<code>/api/products</code>	List all products (newest first)
GET	<code>/api/products/by-ids?ids=id1,id2</code>	Fetch active products by ID array (wishlist)
GET	<code>/api/shipping</code>	Active wilayas with shipping prices
GET	<code>/api/shipping/offices?wilaya=CODE</code>	Delivery offices for a wilaya
GET	<code>/api/gallery</code>	Active gallery images (excludes design- only)
GET	<code>/api/reviews</code>	Approved reviews (max 20)
POST	<code>/api/reviews</code>	Submit new review (status: PENDING)
POST	<code>/api/reviews/upload</code>	Upload review photo (Cloudinary or local)
POST	<code>/api/orders</code>	Create order (validates stock & prices)
POST	<code>/api/contact</code>	Submit contact message
GET	<code>/api/health</code>	Deployment health check
GET	<code>/api/uploads/[...path]</code>	Serve locally uploaded files (dev only)

8.2 Admin-Protected Routes

Method	Route	Description
POST	/api/products	Create product
PUT	/api/products/[id]	Update product
DELETE	/api/products/[id]	Delete product
GET	/api/orders	List all orders
PATCH	/api/orders/[id]	Update order status
GET	/api/reviews?admin=1	List all reviews (including pending)
PATCH	/api/reviews/[id]	Approve/reject review
DELETE	/api/reviews/[id]	Delete review
POST	/api/gallery	Add gallery image
PATCH	/api/gallery/[id]	Update gallery image
DELETE	/api/gallery/[id]	Delete gallery image
GET	/api/contact	List contact messages
POST	/api/admin/login	Admin login
POST	/api/admin/logout	Admin logout
GET	/api/admin/wilayas	List all wilayas with office counts
PATCH	/api/admin/wilayas/[id]	Update wilaya shipping rates
GET	/api/admin/delivery-offices	List offices (optional wilaya filter)
POST	/api/admin/delivery-offices	Create office
PUT	/api/admin/delivery-offices/[id]	Update office
DELETE	/api/admin/delivery-offices/[id]	Delete office

8.3 Health Check

GET /api/health returns:

```
{
  "ok": true,
  "checks": {
    "database": true,
    "admin": true,
    "cloudinary": true
  }
}
```

Returns HTTP 503 if any required check fails. On Vercel, Cloudinary must be configured.

9. Key Code Patterns

9.1 CartContext

File: src/context/CartContext.tsx


```
On mount:
  1. Load cart from localStorage
  2. Fetch /api/products to sync prices & availability
  3. Remove items for inactive/deleted products
  4. Apply current promo prices

On change:
  Save updated cart back to localStorage

Item key: slug + size + color (composite)
```

Exports: addItem, removeItem, updateQuantity, clearCart, total, count, hydrated

9.2 WishlistContext

File: src/context/WishlistContext.tsx

- Stores array of product IDs in localStorage
- toggle(id) adds/removes from wishlist
- has(id) checks membership
- Favorites page fetches full product data via API

9.3 ProductCard Layout

File: src/components/ProductCard.tsx

- Responsive: compact 2-up grid on mobile, expanded on desktop
- Shows promo discount %, "Nouveau" badge for recent items
- Heart icon for wishlist, eye icon for quick view
- Inline size selector before add-to-cart
- Color dots when multiple colors available

9.4 Checkout & Order Creation

Files: PanierClient.tsx, CheckoutForm.tsx, api/orders/route.ts

Client-side validation (validateCheckoutForm):

- Name: min 2 characters
- Phone: Algerian format 05/06/07 + 8 digits
- Instagram: optional, validated handle format
- Wilaya: required
- Delivery method: required
- Address: required for HOME (min 5 chars)
- Office: required for OFFICE pickup

Server-side validation (POST /api/orders):

- Re-fetches products to verify stock and recalculate prices
- Rejects if client-submitted totals don't match (± 1 DA tolerance)
- Decrements stock atomically per item
- Generates unique orderNumber
- Builds formatted address string
- Returns Instagram URL from site settings

Instagram message (buildInstagramOrderMessage):

- Pre-formatted text with emojis
- Customer info, items with sizes/colors, subtotal, shipping, total
- Customer copies message and opens Instagram to send to shop

9.5 Image Handling

Configuration: next.config.ts

```
images: {
  unoptimized: true, // Required on Vercel (/_next/image returns 404)
  remotePatterns: [
    { hostname: "res.cloudinary.com" },
    { hostname: "images.unsplash.com" },
    { hostname: ".*.instagram.com" },
  ],
}
```

Image sources:

Type	Location	Served by
Product images	public/images/*.jpg	Vercel static CDN
Gallery images	DB URL (usually /images/...)	Vercel static CDN
Review photos	Cloudinary secure_url	Cloudinary CDN
Local dev uploads	public/uploads/ or /api/uploads/	Local filesystem

Design-only images: src/lib/images.ts maintains a list of images used only for CSS backgrounds (excluded from gallery display).

9.6 Site Settings & Social Links

Flow:

1. Root layout calls getSocialLinks() server-side
 2. settings.ts reads SiteSettings row from DB
 3. social.ts resolveSocialLinks() merges DB values with DEFAULT_SOCIAL fallbacks
 4. Passed to Providers !' Header, Footer, checkout success screen
-

10. Internationalization (i18n)

10.1 Architecture

Zella Luxe uses a custom i18n solution (no next-intl or i18next):

Layer	File	Role
Translations	src/lib/i18n.ts	All FR/AR strings in translations object
Context	src/context/LanguageContext.tsx	lang, setLang, dir state
Toggle	components/LanguageToggle.tsx	FR !" AR button in header
Product data	DB fields	nameFr/nameAr, descriptionFr/descriptionAr

10.2 Usage Pattern

```
const { lang, dir } = useLanguage();
const tr = t(lang);
// tr.catalogue.title !' "Notre Catalogue" or "bvDdVFb!",bv*"
const name = lang === "fr" ? product.nameFr : product.nameAr;
```

10.3 RTL Support

When Arabic is selected:

- document.documentElement.dir = "rtl"
- document.documentElement.lang = "ar"
- CSS: [dir="rtl"] body { font-family: Noto Sans Arabic, ... }
- Layout components use logical properties and flex direction

10.4 Translation Coverage

All user-facing sections are translated:

- Navigation, hero, catalogue, cart, checkout, wishlist
- Reviews form, contact, gallery, promotions
- Trust bar, splash screen, footer
- Form validation error messages
- Admin panel remains French-only

11. Design System

11.1 Color Palette

Defined in src/app/globals.css CSS variables:

Token	Value	Usage
--bg	#0a0a0d	Page background (near-black)
--foreground	#f4f1ea	Primary text (warm white)
--muted	#a29c92	Secondary text
--gold	#c9a86c	Primary accent (champagne gold)
--gold-light	#e4cf9a	Light gold highlights
--gold-deep	#a07d3e	Deep gold accents

11.2 Typography

Role	Font	Variable
Headings	Playfair Display	--font-display
Body	Jost	--font-sans
Arabic	Noto Sans Arabic	--font-arabic
Script/accent	Great Vibes	--font-script

Loaded via next/font/google in src/app/layout.tsx.

11.3 Visual Effects

- Glass cards: glass-card class — frosted glass with backdrop-blur
- Hero gradient: hero-title-gradient — white-to-gold text gradient
- Background glow: Radial gold gradients via body::before pseudo-element
- Selection: Gold highlight on text selection
- Category accents: Per-category gold shades in CATEGORY_ACCENTS

11.4 Component Conventions

- Rounded corners: rounded-2xl for cards, rounded-full for buttons
- Borders: border-white/10 subtle dividers
- Buttons: Gold gradient CTAs, ghost/outline secondary actions
- Admin: Light theme (admin-shell, admin-card) contrasting with dark storefront
- Mobile-first: Bottom nav, 2-column product grid, safe-area padding

12. Deployment & Operations

12.1 Production Stack

Component	Service	Cost
Hosting	Vercel (Hobby)	Free
Database	Neon PostgreSQL	Free tier (~500 MB)
Image CDN	Cloudinary	Free tier (~25 credits/month)

Official deployment: Vercel + Neon + Cloudinary (see [DEPLOY-VERCEL.md](#))

Render deployment (render.yaml, [DEPLOY.md](#)) is deprecated.

12.2 Build Process

Script: vercel-build in package.json

```
prisma generate && prisma db push --skip-generate && next build
```

Step	Purpose
prisma generate	Generate Prisma client
prisma db push	Sync schema to Neon (create/update tables)
next build	Build optimized production bundle

Note: db push creates tables but does not seed data. Run `npm run db:seed` once after first deploy.

12.3 Initial Data Seeding

```
# Set DATABASE_URL to Neon connection string in .env
npm run db:seed
```

Seeds (prisma/seed.ts + prisma/shipping-seed.ts):

- 58 wilayas with shipping rates
- Delivery offices per wilaya
- Sample luxury products (shoes, bags, luggage)
- Gallery images
- Site settings defaults

Warning: Re-running seed deletes existing products, orders, and reviews.

12.4 Local Development

```
# Option A: Docker PostgreSQL
docker compose up -d
copy .env.example .env
# DATABASE_URL=postgresql://zella:zella@localhost:5432/zellaluxe
npm run db:setup    # push schema + seed
npm run dev

# Option B: Use Neon directly (careful with prod data)
```

Dev server: `next dev --hostname 0.0.0.0` (accessible on LAN)

12.5 Continuous Deployment

Every git push to main triggers automatic Vercel redeployment.

12.6 Monitoring

- Health endpoint: GET /api/health
- Vercel dashboard for build logs and analytics
- Neon dashboard for database metrics

12.7 Data Storage Summary

Data	Storage
Products, orders, reviews, wilayas, messages, settings	Neon PostgreSQL
Review photos	Cloudinary
Product/gallery images (initial)	Git repo (public/images/)
Cart, wishlist, language	Browser localStorage

13. Data Flow Diagrams

13.1 Order Flow

[illegible]

13.2 Review Flow

13.3 Cart Persistence Flow

13.4 Admin Authentication Flow

14. Environment Variables

14. Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string (Neon in prod)
ADMIN_PASSWORD	Yes (prod)	Admin panel login password
CLOUDINARY_CLOUD_NAME	Yes (Vercel)	Cloudinary account cloud name
CLOUDINARY_API_KEY	Yes (Vercel)	Cloudinary API key
CLOUDINARY_API_SECRET	Yes (Vercel)	Cloudinary API secret
UPLOAD_DIR	No (dev only)	Local upload directory override
NODE_ENV	Auto	development or production
VERCEL	Auto	Set by Vercel runtime

See `.env.example` for template. Never commit actual credentials.

15. Appendix: Key File Index

Pages (Server Components)

File	Route
src/app/page.tsx	/
src/app/catalogue/page.tsx	/catalogue
src/app/produit/[slug]/page.tsx	/produit/:slug
src/app/promotions/page.tsx	/promotions
src/app/panier/page.tsx	/panier
src/app/favoris/page.tsx	/favoris
src/app/avis/page.tsx	/avis
src/app/galerie/page.tsx	/galerie
src/app/contact/page.tsx	/contact
src/app/admin/page.tsx	/admin
src/app/admin/login/page.tsx	/admin/login

Configuration

File	Purpose
package.json	Dependencies, npm scripts
next.config.ts	Image config, remote patterns
vercel.json	Vercel service routing
prisma/schema.prisma	Database schema
docker-compose.yml	Local PostgreSQL
DEPLOY-VERCEL.md	Deployment guide
.env.example	Environment template

npm Scripts

Script	Command	Purpose
dev	next dev --hostname 0.0.0.0	Development server
build	prisma generate && next build	Production build
vercel-build	prisma generate && prisma db push && next build	Vercel deploy build
db:push	prisma db push	Sync schema to DB
db:seed	tsx prisma/seed.ts	Seed initial data
db:setup	db:push && db:seed	Full local DB setup
db:studio	prisma studio	Visual DB browser

End of Zella Luxe Technical Documentation

Generated from source code analysis — July 2026

